1 Predlog projekta: Sistem za preporuku championa u League of Legends korišćenjem rule-based pristupa (Drools)

1.1 Članovi tima

- Filip Vidaković

1.2 Opis problema

1.2.1 Motivacija

U igrama kao što je League of Legends, izbor odgovarajućeg championa u fazi selekcije (champion select) ima značajan uticaj na ishod partije. Igrači, naročito oni sa manje iskustva, često donose odluke bez dovoljno razumevanja kompozicije timova, sinergije između championa i potencijalnih counter pick-ova.

Cilj ovog projekta je razvoj sistema baziranog na pravilima koji simulira način razmišljanja iskusnog igrača i pruža preporuku championa na osnovu sastava oba tima.

1.2.2 Pregled problema i literatura

Postojeći alati za pomoć pri izboru championa (npr. statističke platforme poput OP.GG ili U.GG) uglavnom se zasnivaju na:

- winrate statistikama
- popularnosti championa
- istorijskim matchup podacima

Međutim, nedostaci ovih rešenja su:

- nedostatak objašnjenja (zašto je nešto preporučeno)
- ignorisanje konteksta trenutne kompozicije tima
- fokus na pojedinačne matchup-ove umesto na celokupnu timsku dinamiku

Predloženo rešenje se razlikuje po tome što:

- koristi **eksplicitno definisana pravila (rule-based sistem)**
- omogućava **transparentno rezonovanje**
- uzima u obzir **više faktora istovremeno** (balans tima, tip štete, stil igre)

Prednost sistema je u tome što može da objasni svoju odluku kroz niz zaključaka, čime se simulira ekspertno znanje.

1.3 Metodologija rada

1.3.1 Ulaz u sistem (input)

Sistem prima sledeće ulazne podatke:

- Lista championa u neprijateljskom timu
- Lista championa u sopstvenom timu
- Atributi championa:
    - uloga (tank, assassin, mage, ADC, support)
    - tip štete (AD/AP)
    - stil igre (burst, sustain, engage, poke)

Ulaz se modeluje kroz objekte tipa:

- Champion
- Team

1.3.2 Izlaz iz sistema (output)

Sistem generiše:

- Jednu ili više preporuka championa
- Objašnjenje preporuke kroz zaključene činjenice i strategije

Primer izlaza:

- Preporučeni champion: Malphite
- Objašnjenje:

    - neprijateljski tim ima dominantno AD štetu
    - prisutan je assassin
    - timu nedostaje tank

1.3.3 Baza znanja

Baza znanja sastoji se od:

- Skupa definisanih championa (~30 reprezentativnih)
- Njihovih atributa
- Pravila koja definišu odnose između podataka

Struktura znanja:

1. **Činjenice**
    - Champion (atributi)
    - Team (sastav timova)
2. **Međučinjenice (zaključci)**
    - enemyMostlyAD
    - enemyHasAssassin
    - teamNeedsTank
    - teamNeedsAP
3. **Strategije**
    - pickTank
    - buildArmor
    - pickMage
4. **Preporuke**
    - konkretni championi

Popunjavanje baze znanja:

- Ručno definisani atributi championa

- Pravila zasnovana na:
    - iskustvu i domenskom znanju inženjera
    - javno dostupnim vodičima i matchup analizama

Interakcija:

- Korisnik unosi sastav timova
- Sistem aktivira pravila (forward chaining)
- Generišu se zaključci i preporuke

Pravila:

- Analiza enemy tima:
    - enemyMostlyAD
    - enemyMostlyAP
    - enemyHasAssasin
    - enemyHasTank
- Analiza korisnikovog tima:
    - teamNeedsTank
    - teamNeedsAp
    - teamNeedsEngage
- Kombinacije (strategije):
    - pickTank
    - buildArmor
- Pravila za preporuku: pickTank + buildArmor = Malphite
- CEP pravila
    - enemyTrendingAD
    - enemyAgressiveDraft
- BC queries – isGoodPick(champion)

1.4 Primer rezonovanja (korak po korak)

Ulaz:

- Enemy tim: Zed, Talon, Caitlyn
- My tim: Garen, Ashe

Koraci:

1. Sistem identifikuje:
    - Zed i Talon → assassin
    - sva tri championa → AD damage
2. Aktivira pravila:
    - "enemyMostlyAD"
    - "enemyHasAssassin"
3. Izvodi zaključke:
    - potreban armor
    - potreban tank
4. Generiše strategiju:
    - pickTank
    - buildArmor
5. Konačna preporuka:
    - Malphite

1.5 Primeri kompleksnih pravila

Primer 1 – Korišćenje accumulate funkcije

```
rule "Enemy mostly AD"
when
   accumulate(
      Champion(team == "enemy", damageType == "AD"),
      $count : count(1)
   )
   eval($count >= 3)
then
   insert(new Fact("enemyMostlyAD"));
```

```
end
```

Primer 2 – Ulančavanje pravila (3 nivoa)

```
rule "Enemy has assassin"
when
   Champion(team == "enemy", role == "assassin")
then
   insert(new Fact("enemyHasAssassin"));
end
rule "Need tank"
when
   Fact(name == "enemyHasAssassin")
then
   insert(new Strategy("pickTank"));
end
rule "Recommend Malphite"
when
   Strategy(name == "pickTank")
   Fact(name == "enemyMostlyAD")
then
   insert(new Recommendation("Malphite"));
end
```

1.6 Napredne funkcionalnosti (za ocenu 10)

1.6.1 CEP (Complex Event Processing)

Sistem će analizirati sekvence događaja:

- npr. više uzastopnih izbora championa istog tipa
- vremenski ograničeni događaji (npr. draft faza)

Primer:

- ako protivnik u kratkom vremenu izabere 3 AD championa - aktivira se specifično pravilo

Svaki Pick predstavlja jedan dogadjaj. Pamti se champion, team (my/enemy), timestamp, role, damageType. Na osnovu ovoga možemo primetiti šablon, npr agresivno pickovanje

---

1.6.2 Backward chaining

Sistem može odgovarati na pitanja tipa:

- "Da li je Malphite dobar izbor?"

Proces:

- proverava uslove koji moraju biti ispunjeni
- analizira da li postoje odgovarajuće činjenice

Na primer:

IsGoodPick(Malphite):

- Strategy(pickTank)
    - Fact(enemyHasAssasin)
        - Champion(role = assasin, team = enemy)
- Fact(enemyMostlyAd)
    - Accumulate(Champion(damageType = AD, team = enemy)) >= 3

1.6.3 Template pravila

Korišćenje template-a za generisanje pravila za više championa:

- smanjenje redundanse
- lakše održavanje sistema

Neki od parametara: championName, requiredRole, requiredDamageType, ...

1.7 Zaključak

Predloženi sistem predstavlja rule-based pristup rešavanju problema izbora championa u League of Legends. Fokus projekta je na modelovanju znanja i implementaciji višeslojnog rezonovanja kroz Drools.

Sistem omogućava:

- transparentno donošenje odluka
- objašnjive preporuke
- fleksibilno proširenje baze znanja